

Lab 1: Identify pipeline components from a scenario

Orders DB



Ingestion: CDC → Kafka



Storage: Raw landing zone (Parquet)



Processing: Batch ETL (Spark)



Serving: Data warehouse (Redshift) → BI dashboards



Monitoring: Airflow + Prometheus

Daily product catalog CSV files arrive from vendors:

Product catalog CSV



Ingestion: Batch → Whenever vendors upload new files



Storage: Data lake, long term and cheap storage



Processing: Batch ETL (Spark)



Serving: Data warehouse → dashboards



Monitoring: Data quality, job failures

User clicks are logged to an event log collection system:

Clicks



Ingestion: Streaming using Kafka



Storage: Data Warehouse, optimized for analytics and fast queries.



Processing: Streaming (Kafka) ELT



Serving: OLAP tables → dashboards/reports



Monitoring: Ingestion latency, Job failures, Data volume variance

Lab 2: Classify systems as OLTP or OLAP

System	Classification	Reason
Customer profile database used by customer support for updates.	OLTP	Data is frequently updated
Daily sales reporting database used by analysts for trend analysis.	OLAP	Large amounts of historical data used for analysis.
Session store for active web sessions.	OLTP	Small records that has to be written
Historical analytics store for machine learning training.	OLAP	Large amounts of historical data used for analysis rather than frequent updates.

Lab 3: Batch vs Streaming: classify processing

Task	Batch/Streaming	Reason	Suggested tool
Generate daily sales summary for dashboard (end-of-day).	Batch	The latency is daily and can be processed at once. It is used for dashboards and reporting	Spark/Airflow
Detect fraudulent transactions within 5 seconds.	Streaming	The latency is short, seconds after the event. It is for alerting	Kafka/Flink
Aggregate hourly user activity for retention metrics.	Batch	The data is processed once per hour for backfilling	Spark/Airflow
Enrich clickstream events with user profile in near-real-time.	Streaming	The latency is sort, seconds after the event. It is for enrich and personalize real time data	Kafka/Flink

Lab 4: ETL vs ELT decision for a use case

Use case: You need to bring vendor-supplied CSV product feeds into your analytics.

Files have inconsistent columns and often include new fields. Decide whether ETL or ELT is better. Provide a short plan for the chosen approach.

ELT process. The data is provided by external vendors and could have new fields and with ELT it is easier to reprocess the files and backfill. The CSV are loaded in raw and after transformed depending on the file. The ELT has more flexibility.

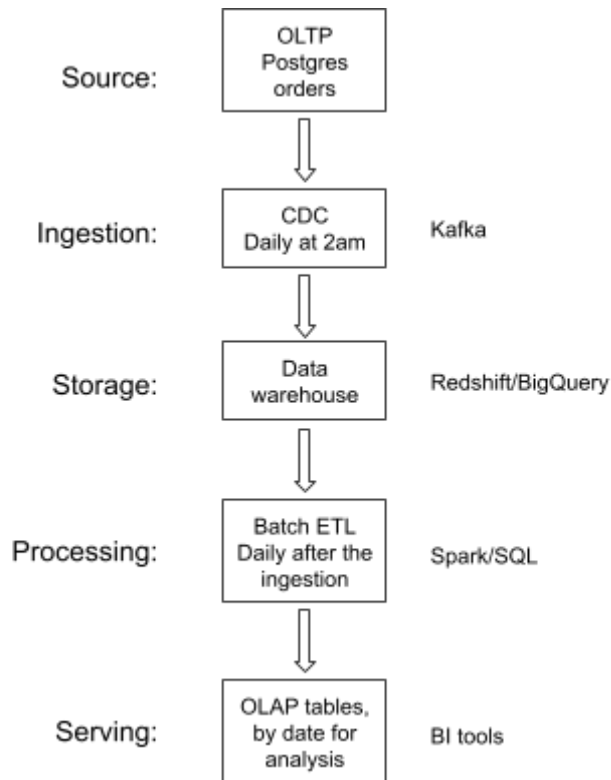
- Ingestion: Batch ingestion when the file is delivered.
- Storage: Object store, data lake. Each CSV is stored as a raw object in the data lake.
- Transform: Read the file, check the columns, add the new fields, standardize inconsistent columns across CSV files, deal with missing values.
-

Lab 5: Map sources to ingestion, storage, processing, serving

Source	Ingestion	Landing zone	Processing	Serving example
Relational orders DB (Postgres)	CDC	Raw	Batch	OLAP tables
Vendor CSV product files	Batch	Raw	Batch	Warehouse
Mobile app events (high-volume)	Streaming	Bronze	Streaming	OLAP tables
Product catalog in DynamoDB	CDC	Raw	Batch	Warehouse

Lab 6: Design a batch e-commerce pipeline (orders)

Design a batch pipeline to move daily orders from an OLTP Postgres orders table into an analytics schema in a data warehouse. Include components, schedule, file format, partitioning, and a simple transformation plan.



List of chosen technologies:

- CDC ingestion
- Kafka remove duplicated orders in near real time
- Data warehouse like Redshift or BigQuery to storage the data.
- Spark and SQL for data transformation
- OLAP tables for BI Tools
-

Example SQL for one transformation (daily revenue per product)

```
SELECT
    date,
    product,
    sum(amount) as revenue
FROM orders
GROUP BY date, product
```

Lab 7: Design a streaming user activity pipeline (clickstream)

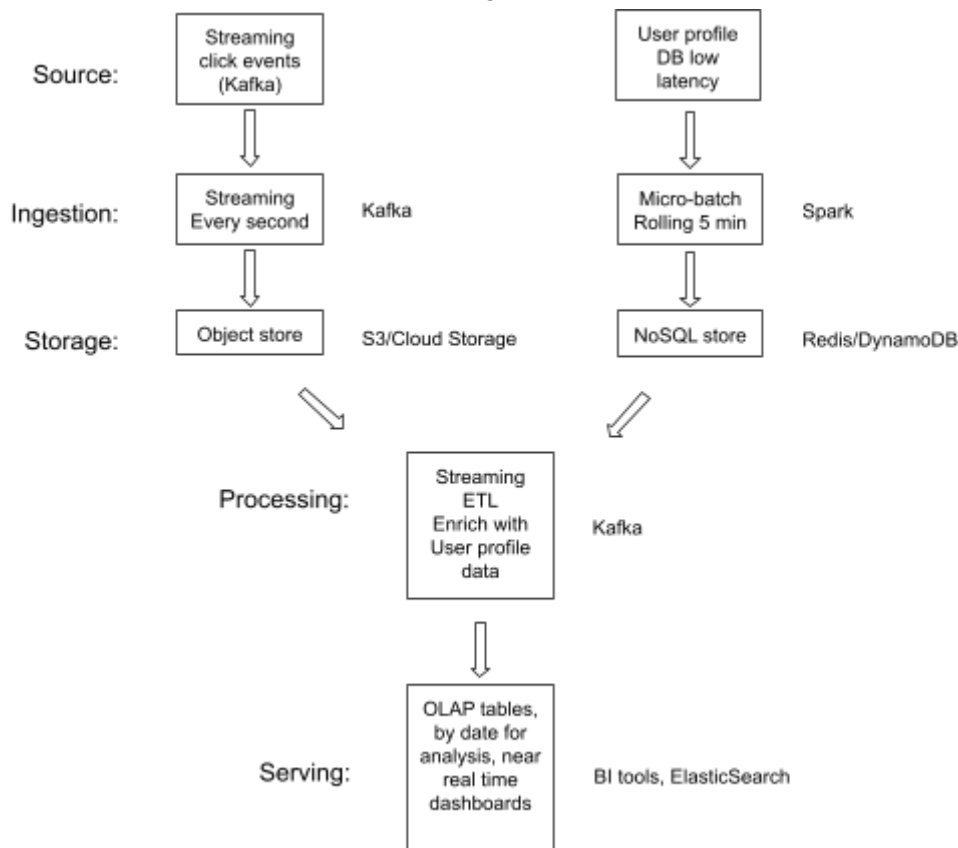
Design a streaming pipeline that collects click events from a web app, enriches events with user profile info, computes a rolling 5-minute unique-user metric, and serves to a near-real-time dashboard.

Scenario / Input

Click events arrive at ~10k events/sec.

User profile store is in a low-latency key-value DB (Redis or DynamoDB).

Dashboard requires < 1-minute latency for metrics.



Key architectural choices:

Ingesting the events with Kafka because of the low latency (10k events per second). Use Kafka for real-time ingestion and a stream processing engine to enrich the data with user profile information.

Pseudocode/Python snippet showing event enrichment and rolling unique users (small example):

```
# web app events collection
```

```
events=[{
    "event_id",
    "event_name",
    "timestamp",
```

```

        "user_id"
    ]]

# user profile collection
user_profile = [{
    "user_id": string,
    "user_device": string,
    "user_registered": boolean
    "timestamp": timestamp
}]

# user metric for 5 min rolling
user_metric(user_profile, duration="5 minutes")

# events enrichment
enriched_event = event.join(user_metric, on=user_id, how=left)

# unique users
unique_users = count_distinct(enriched_event["user_id"])

```

Lab 8: Choose ingestion tools for APIs, files, and logs

Source	Tool	Why	Notes on reliability & retries
Third-party REST API with hourly updates (rate-limited).	Airflow	It can schedule the running jobs, retry if a task fails and track the status of every job	Airflow has configurable retries, retry delays and task status monitoring
Daily FTP drop of large CSVs (multi-GB).	Spark	Batch processing for large amounts of data that allows efficient reading and processing in parallel	Spark can restart failed jobs
High-volume application logs written to files on many servers.	Kafka	Can process huge amounts of real time data	Kafka persists the messages